

TypeScript & Next.js

Interview Questions & Answers

A curated set of commonly asked interview questions covering TypeScript fundamentals through advanced type system features, and Next.js from routing basics through rendering strategies, data fetching, and performance.

Part 1 — TypeScript

1. Basics & Core Concepts
2. Types, Interfaces & Type Aliases
3. Advanced Types (Generics, Utility Types, Conditional Types)
4. OOP in TypeScript (Classes, Access Modifiers, Abstract Classes)
5. Type Narrowing, Guards & Assertions
6. Modules, Configuration & Tooling
7. Common Coding / Practical Questions

Part 2 — Next.js

1. Fundamentals
2. Routing (App Router & Pages Router)
3. Rendering Strategies (SSR, SSG, ISR, CSR)
4. Data Fetching
5. API Routes & Server Actions
6. Performance & Optimization
7. Middleware, Auth & Deployment
8. Common Coding / Practical Questions

PART 1: TYPESCRIPT

1. Basics & Core Concepts

Q1. What is TypeScript and how is it different from JavaScript?

TypeScript is a statically typed superset of JavaScript developed by Microsoft that compiles down to plain JavaScript. It adds optional static typing, interfaces, generics, and compile-time type checking, which JavaScript lacks since JS is dynamically typed and checks types only at runtime.

Q2. What are the benefits of using TypeScript?

Early error detection at compile time, better IDE support (autocomplete, refactoring, navigation), improved code readability and self-documentation via types, easier large-scale codebase maintenance, and better collaboration in teams.

Q3. What are the basic types in TypeScript?

string, number, boolean, null, undefined, void, any, unknown, never, bigint, symbol, arrays (type[] or Array<type>), tuples, and enums.

Q4. What is the difference between 'any' and 'unknown'?

'any' disables type checking entirely and allows any operation on the value. 'unknown' is a type-safe counterpart — you can assign anything to it, but you must narrow its type (via type guards or assertions) before performing operations on it.

Q5. What is the difference between 'null' and 'undefined' in TypeScript?

'undefined' means a variable has been declared but not assigned a value. 'null' is an intentional absence of value. With strictNullChecks enabled, TypeScript treats them as distinct types and won't allow assigning one where the other isn't expected.

Q6. What is the 'never' type and when is it used?

'never' represents values that never occur — used for functions that always throw an error or have infinite loops, and for exhaustiveness checks in switch statements over union types.

Q7. What does the 'strict' compiler flag do?

It enables a group of strict type-checking options at once, including noImplicitAny, strictNullChecks, strictFunctionTypes, strictBindCallApply, strictPropertyInitialization, noImplicitThis, and alwaysStrict — producing much safer code.

2. Types, Interfaces & Type Aliases

Q8. What is the difference between 'interface' and 'type'?

Both describe object shapes. Interfaces can be reopened and merged (declaration merging) and are generally preferred for public API object shapes; type aliases can represent unions, tuples, primitives, and mapped/conditional types, and cannot be merged after declaration. In most simple object cases they're interchangeable.

Q9. What are union and intersection types?

A union type (A | B) means a value can be either type A or type B. An intersection type (A & B) combines multiple types into one that must satisfy all of them simultaneously.

Q10. What is a tuple in TypeScript?

An array with a fixed number of elements where each element's type is known and fixed at a specific position, e.g. let x: [string, number] = ['age', 25].

Q11. What are enums? What's the difference between numeric and string enums?

Enums let you define a set of named constants. Numeric enums auto-increment from 0 (or a specified start) by default; string enums require each member to be initialized with a string literal, which makes debugging clearer since values are self-descriptive.

Q12. What are literal types?

Types that represent an exact value rather than a general type, e.g. let dir: 'up' | 'down' restricts the variable to exactly those two string literals.

Q13. What is declaration merging?

TypeScript's ability to combine two or more separate declarations with the same name into a single definition — most commonly used with interfaces, where multiple interface declarations with the same name are automatically merged into one.

3. Advanced Types

Q14. What are generics and why are they useful?

Generics allow writing reusable, type-safe components/functions that work over a variety of types rather than a single one, while preserving type information — e.g. function identity<T>(arg: T): T. They avoid the loss of type safety that 'any' would introduce.

Q15. What are generic constraints?

A way to restrict the types a generic parameter can accept, using 'extends', e.g. function longest<T extends {length: number}>(a: T, b: T): T. This ensures the generic type has certain properties/methods available.

Q16. What are utility types? Name a few commonly used ones.

Built-in generic types that transform existing types. Common ones: Partial<T> (makes all properties optional), Required<T>, Readonly<T>, Pick<T, K>, Omit<T, K>, Record<K, T>, Exclude<T, U>, Extract<T, U>, ReturnType<T>, and Parameters<T>.

Q17. What are mapped types?

Types that create new types by transforming each property of an existing type, typically using 'in keyof', e.g. type Readonly<T> = { readonly [P in keyof T]: T[P] }.

Q18. What are conditional types?

Types expressed in the form 'T extends U ? X : Y', which choose one of two types based on a condition evaluated at the type level. Combined with 'infer', they can extract types from within other types, as ReturnType<T> does internally.

Q19. What is the 'infer' keyword used for?

Used inside conditional types to declare a type variable to be inferred from within the type being checked, e.g. extracting the return type of a function type.

Q20. What is the difference between 'keyof' and 'typeof' operators?

'keyof' produces a union of the property names (keys) of a given type. 'typeof', when used in a type context, extracts the static type of a variable or object, letting you derive a type from an existing value.

Q21. What are template literal types?

Types built using template string syntax that can combine and manipulate string literal types, e.g. type Greeting = `Hello, \${string}`, useful for generating precise string-based type unions.

Q22. What is function overloading in TypeScript?

Defining multiple call signatures for the same function name so it can be called in different ways with different parameter/return types, with the compiler picking the matching overload signature at each call site.

4. OOP in TypeScript

Q23. What access modifiers does TypeScript support?

public (default, accessible everywhere), private (accessible only within the declaring class), and protected (accessible within the class and its subclasses). TypeScript also supports the readonly modifier and native JS '#' private fields.

Q24. What is an abstract class and how does it differ from an interface?

An abstract class can provide partial implementation and cannot be instantiated directly — subclasses must implement its abstract members. Unlike an interface, it can contain actual method implementations, constructors, and field values, not just shape declarations.

Q25. Can a class implement multiple interfaces?

Yes — unlike classes (which support only single inheritance via 'extends'), a class can implement multiple interfaces using a comma-separated list after 'implements'.

Q26. What are parameter properties in constructors?

A shorthand where adding an access modifier (public/private/protected/readonly) directly to a constructor parameter automatically creates and initializes a matching class property, avoiding repetitive assignment code.

Q27. What is the difference between an interface extending another interface and a class implementing an interface?

Interface-to-interface extension inherits and combines type shapes. A class implementing an interface commits to providing concrete implementations for every member declared in that interface.

5. Type Narrowing, Guards & Assertions

Q28. What is type narrowing?

The process by which TypeScript refines a variable's type to a more specific type within a certain code branch, based on checks like typeof, instanceof, in, equality checks, or custom type guard functions.

Q29. What is a user-defined type guard?

A function whose return type is a type predicate (e.g. 'function isFish(pet: Fish | Bird): pet is Fish'), which TypeScript uses to narrow a union type within conditional blocks that call it.

Q30. What is the difference between type assertion ('as') and type casting?

TypeScript doesn't perform runtime casting/conversion — 'as' (or angle-bracket syntax) is purely a compile-time assertion telling the compiler to treat a value as a specific type, without any runtime check or conversion actually happening.

Q31. What is the non-null assertion operator ('!')?

A postfix operator that tells the compiler a value is definitely not null or undefined at that point, suppressing the corresponding compile-time error — it performs no runtime check, so misuse can still cause runtime errors.

Q32. What is discriminated union (tagged union) pattern?

A pattern where each type in a union shares a common literal property (the 'discriminant', e.g. 'kind') with a distinct value per variant, allowing TypeScript to narrow the union precisely based on that property in switch/if

checks.

6. Modules, Configuration & Tooling

Q33. What is the purpose of `tsconfig.json`?

It configures the TypeScript compiler's behavior for a project — target JS version, module system, strictness flags, output directory, included/excluded files, path aliases, and more.

Q34. What is the difference between 'interface' merging and module augmentation?

Declaration merging combines multiple interface declarations of the same name. Module augmentation extends the shape of an existing module (including third-party or global types) from a different file, commonly used to add custom properties to libraries like Express's Request object.

Q35. What are ambient declarations (`.d.ts` files)?

Files that describe the shape of existing JavaScript code (types only, no implementation) so TypeScript can type-check code that consumes plain JS libraries lacking their own type definitions.

Q36. What is the difference between 'import type' and a regular 'import'?

'import type' explicitly imports only type information, which is guaranteed to be erased at compile time and never produces a runtime import — useful for avoiding unnecessary bundling and circular import issues.

Q37. What does the 'target' and 'lib' option in `tsconfig` control?

'target' sets which ECMAScript version the TypeScript compiler emits JS for (e.g. ES2015, ES2020). 'lib' specifies which built-in type declaration files (DOM, ES2020, etc.) are available to the type checker.

7. Common Coding / Practical Questions

Q38. Write a generic function that returns the last element of an array.

`function last<T>(arr: T[]): T | undefined { return arr[arr.length - 1]; }` — demonstrates a generic constrained naturally by the array element type, with a safe undefined fallback for empty arrays.

Q39. How would you type a function that takes either a string or number and returns a boolean, handling each differently?

Use a union parameter type combined with a `typeof` type guard inside the function body: `function check(val: string | number): boolean { if (typeof val === 'string') { ... } else { ... } }`.

Q40. How do you make all properties of an interface optional except one?

Combine utility types: `type PartialExceptId<T, K extends keyof T> = Partial<Omit<T, K>> & Pick<T, K>`, which keeps K required while making the rest optional.

Q41. What will `'const x: number = null'` do under `strictNullChecks`, and why?

It produces a compile error, because with `strictNullChecks` enabled 'null' is not assignable to 'number' — null/undefined must be explicitly included in a type's union to be allowed.

Q42. How does TypeScript handle excess property checks on object literals?

When assigning an object literal directly to a typed variable or parameter, TypeScript flags any extra properties not defined on the target type as errors — this check doesn't apply if the object comes from a variable rather than a literal.

PART 2: NEXT.JS

1. Fundamentals

Q1. What is Next.js and why use it over plain React?

Next.js is a React framework that adds file-based routing, server-side rendering, static site generation, API routes, image/font optimization, and a build/bundling pipeline out of the box, removing the need to hand-assemble tooling like React Router, Webpack config, and a Node server yourself.

Q2. What is the difference between the App Router and the Pages Router?

The Pages Router (pages/ directory) is the original file-based routing system using React components per route. The App Router (app/ directory), introduced in Next.js 13+, adds React Server Components by default, nested layouts, streaming, and colocated loading/error states, and is now the recommended approach.

Q3. What are Server Components and Client Components?

Server Components (default in the App Router) render on the server, can access backend resources directly, and ship no JS to the client. Client Components (marked with the 'use client' directive) render interactive UI that needs browser APIs, state, or event handlers, and are hydrated on the client.

Q4. What problem does Next.js solve compared to a client-only React SPA?

It solves slow initial load and poor SEO in single-page apps by rendering content on the server (or at build time), sending fully-formed HTML to the browser, while still enabling rich client-side interactivity after hydration.

Q5. What is hydration in the context of Next.js?

The process where React attaches event listeners and internal state to the server-rendered HTML already sent to the browser, turning static markup into a fully interactive React application without re-rendering from scratch.

2. Routing

Q6. How does file-based routing work in Next.js?

Each file (or folder with a page.tsx/js file, in the App Router) inside the app/ or pages/ directory automatically becomes a route matching its file path, without needing a separate router configuration.

Q7. How do you create dynamic routes?

By wrapping a folder or filename in square brackets, e.g. app/blog/[slug]/page.tsx or pages/blog/[slug].js, where the bracketed segment becomes an accessible route parameter.

Q8. What are catch-all and optional catch-all routes?

A catch-all route uses [...slug] to match any number of path segments as an array. An optional catch-all route uses [...slug] and additionally matches the route with zero segments (the parent route itself).

Q9. What are route groups in the App Router?

Folders wrapped in parentheses, e.g. (marketing), that organize routes or apply shared layouts without affecting the actual URL path.

Q10. What is a layout.tsx file and how does nested layout composition work?

layout.tsx defines UI shared across a route segment and its children, persisting across navigations without re-rendering. Layouts nest automatically based on folder hierarchy, so a child route's layout wraps inside its parent's layout.

Q11. What are parallel and intercepting routes?

Parallel routes (using `@slot` folders) render multiple independent pages simultaneously within the same layout, useful for dashboards. Intercepting routes (using `(.) / (..)` conventions) let a route load a different route within the current layout, commonly used for modals.

Q12. How do you programmatically navigate in Next.js?

In the App Router, using the `useRouter` hook from 'next/navigation' (`router.push`, `router.replace`) inside Client Components, or the `<Link>` component for declarative navigation. In the Pages Router it's 'next/router' instead.

3. Rendering Strategies

Q13. What is Server-Side Rendering (SSR) in Next.js?

Rendering a page to HTML on the server for every incoming request, ensuring the content is always fresh — done via `async` Server Components in the App Router, or `getServerSideProps` in the Pages Router.

Q14. What is Static Site Generation (SSG)?

Rendering pages to HTML at build time so they can be served as static files from a CDN — the fastest strategy for content that doesn't change per request, done via `getStaticProps` in the Pages Router, or by default for Server Components without dynamic data in the App Router.

Q15. What is Incremental Static Regeneration (ISR)?

A hybrid approach that lets statically generated pages be regenerated in the background after a specified revalidation interval, without needing a full rebuild — configured via the 'revalidate' option.

Q16. How do SSR, SSG, ISR and CSR differ, and when would you choose each?

SSG is best for mostly-static content (marketing pages, blogs) for maximum performance; ISR suits content that changes occasionally (product listings); SSR suits pages needing per-request freshness or personalization (dashboards); CSR (client-side rendering) suits highly interactive, user-specific UI where SEO isn't a priority.

Q17. What is streaming and how does Next.js support it?

Streaming sends UI to the client progressively as it becomes ready rather than waiting for the entire page, implemented in the App Router via React Suspense boundaries and the special `loading.tsx` file for route segments.

Q18. What is the difference between static and dynamic rendering in the App Router?

Static rendering happens at build time and is cached; dynamic rendering happens at request time. A route becomes dynamic automatically if it uses dynamic functions (`cookies()`, `headers()`) or uncached data requests, or is explicitly opted in via `'export const dynamic = force-dynamic'`.

4. Data Fetching

Q19. How do you fetch data in the App Router?

By calling `fetch` (or a database/ORM call) directly inside an `async` Server Component, since Server Components can be `async` functions and `await` data before rendering, without needing `useEffect` or client-side fetching libraries.

Q20. How does Next.js extend the native fetch API for caching?

It augments `fetch` with a 'cache' option ('force-cache' for static caching, 'no-store' for always-fresh data) and a 'next: { revalidate }' option to control time-based revalidation per request.

Q21. What is `getStaticProps` and when does it run?

A Pages Router data-fetching function that runs at build time (and optionally on a revalidation interval) to supply props to a statically generated page.

Q22. What is `getServerSideProps` and when does it run?

A Pages Router function that runs on every request on the server before rendering the page, used when data must be fresh for each request.

Q23. What is `getStaticPaths` used for?

In the Pages Router, it defines which dynamic route parameters should be pre-rendered at build time when using `getStaticProps` on a dynamic route, along with a fallback strategy for paths not listed.

Q24. How do you handle client-side data fetching and caching in Next.js apps?

Commonly via libraries like SWR or TanStack Query inside Client Components, which provide caching, revalidation, and request deduplication for data that must be fetched or updated after the initial page load.

5. API Routes & Server Actions

Q25. What are API routes in Next.js?

Files inside `app/api/.../route.ts` (App Router) or `pages/api/` (Pages Router) that let you build backend HTTP endpoints within the same Next.js project, avoiding a separate backend server for simple needs.

Q26. What are Server Actions?

Async functions marked with 'use server' that can be called directly from Client or Server Components (including as form action handlers) to perform server-side mutations without manually building an API route.

Q27. How do you handle form submissions with Server Actions?

By passing a Server Action function directly to a form's 'action' prop; Next.js handles the request, and the `useFormState/useActionState` and `useFormStatus` hooks can track pending state and validation errors on the client.

Q28. How do you revalidate cached data after a mutation?

Using `revalidatePath()` or `revalidateTag()` from 'next/cache' inside a Server Action or Route Handler, which purges the relevant cache entries so subsequent requests get fresh data.

6. Performance & Optimization

Q29. How does `next/image` optimize images?

It automatically resizes, compresses, and serves images in modern formats (like WebP), lazy-loads offscreen images by default, and prevents layout shift by requiring width/height (or using fill), all without manual image processing.

Q30. How does `next/font` optimize font loading?

It self-hosts fonts (including Google Fonts) at build time, eliminating extra network requests to third-party font servers and avoiding layout shift from font-swapping (FOUT/FOIT).

Q31. What is code splitting and how does Next.js handle it automatically?

Code splitting breaks the JS bundle into smaller chunks loaded on demand. Next.js automatically splits code per route, so each page only loads the JavaScript it needs.

Q32. What is dynamic import (`next/dynamic`) used for?

Lazily loading a component (often a heavy or client-only one) only when it's needed, optionally disabling server-side rendering for it via `{ssr: false}`, reducing initial bundle size.

Q33. What tools does Next.js provide to analyze bundle size?

The `@next/bundle-analyzer` plugin visualizes what's inside each JavaScript bundle, helping identify large dependencies worth code-splitting or replacing.

7. Middleware, Auth & Deployment

Q34. What is Next.js Middleware and where does it run?

Code defined in a `middleware.ts` file at the project root that runs before a request is completed, at the edge, allowing you to rewrite, redirect, or modify requests/responses (e.g. for auth checks or A/B testing) based on cookies, headers, or geolocation.

Q35. How is authentication typically handled in Next.js apps?

Via session/JWT cookies checked in Middleware or Server Components for route protection, often using libraries like `NextAuth.js/Auth.js`, or a third-party provider (Clerk, Auth0), combined with Server Actions for login/logout mutations.

Q36. What is the difference between Edge and Node.js runtimes in Next.js?

The Edge runtime is a lightweight, fast-starting JS runtime with a restricted API surface (no native Node APIs) suited to Middleware and latency-sensitive logic run close to users. The Node.js runtime supports the full Node API surface for more complex server-side work.

Q37. What is the difference between 'next build' output modes (server vs 'output: export')?

The default build produces a Node.js server capable of SSR/ISR/API routes. 'output: export' produces a fully static HTML export with no server, suitable only for apps that don't need SSR, ISR, API routes, or Image Optimization at runtime.

Q38. How do environment variables work in Next.js?

Variables in `.env` files are available server-side by default; only variables prefixed with `NEXT_PUBLIC_` are inlined into the client-side JavaScript bundle at build time and thus exposed to the browser.

8. Common Coding / Practical Questions

Q39. How would you create a dynamic blog route that fetches a post by slug in the App Router?

Create `app/blog/[slug]/page.tsx` as an async Server Component: `export default async function Page({ params }) { const post = await getPost(params.slug); return <Article post={post} />; }` — data fetching happens directly in the component.

Q40. How would you protect a set of routes so only authenticated users can access them?

Add a `middleware.ts` with a 'matcher' config targeting those paths; inside it, check the session cookie/token and use `NextResponse.redirect()` to send unauthenticated users to a login page before the route ever renders.

Q41. How would you show a loading skeleton while a slow data-fetching Server Component resolves?

Add a `loading.tsx` file alongside the route's `page.tsx` (App Router) — Next.js automatically wraps the route in a `Suspense` boundary using that file as the fallback UI while the async component streams in.

Q42. How would you revalidate a static page after a CMS webhook fires on content update?

Create an API route (or Route Handler) that verifies the webhook request, then calls `revalidatePath('/blog/[slug]')` or `revalidateTag('posts')` to purge the cached page so the next request regenerates it with fresh content.

Tip: Practice explaining these concepts out loud and pair each answer with a small code example — interviewers often ask you to sketch a snippet on a whiteboard or in a shared editor as a follow-up.